

AppealNet: An Efficient and Highly-Accurate Edge/Cloud Collaborative Architecture for DNN Inference

Min Li*, Yu Li*, Ye Tian*, Li Jiang[†] and Qiang Xu*

**CUhk REliable Computing Laboratory (CURE),*

Department of Computer Science and Engineering, The Chinese University of Hong Kong, Shatin, Hong Kong S.A.R.

{mli, yuli, tianye, qxu}@cse.cuhk.edu.hk

[†]*Department of Computer Science and Engineering, MoE Key Lab of Artificial Intelligence, AI Institute, Shanghai Jiao Tong University, Shanghai, China*

Abstract—This paper presents *AppealNet*, a novel edge/cloud collaborative architecture that runs deep learning (DL) tasks more efficiently than state-of-the-art solutions. For a given input, *AppealNet* accurately predicts on-the-fly whether it can be successfully processed by the DL model deployed on the resource-constrained edge device, and if not, appeals to the more powerful DL model deployed at the cloud. This is achieved by employing a two-head neural network architecture that explicitly takes inference difficulty into consideration and optimizes the tradeoff between accuracy and computation/communication cost of the edge/cloud collaborative architecture. Experimental results on several image classification datasets show up to more than 40% energy savings compared to existing techniques without sacrificing accuracy.

I. INTRODUCTION

Over the years, internet-of-things (IoTs) have become increasingly intelligent by employing deep neural network (DNN) models for learning-based processing. Due to the stringent resource constraints of such edge devices, the DNN models used in them must be small. To address this problem, various DNN model compression techniques (e.g., *weight pruning* [1], *quantization* [2], *knowledge distillation* [3]) and compact DNN architectures [4] were proposed for edge computing in the literature.

Small DNN models are usually less accurate than large ones due to their limited model capacities. Generally speaking, small models have difficulty handling those corner cases that lie in the long-tail of data distribution, but they can still infer correctly for the majority of the other inputs. Consequently, it is possible to employ an edge/cloud collaborative architecture that processes those ‘easy’ inputs with a small DNN model at edge and offloads ‘difficult’ inputs to the powerful DNN model at cloud [5]–[8]. Such an architecture facilitates to achieve a good energy-accuracy tradeoff for DNN inference when compared to edge-only or cloud-only solutions.

No doubt to say, the effectiveness of such an edge/cloud collaborative architecture depends on the accuracy of the *predictor* that is responsible for differentiating ‘easy’ and ‘difficult’ inputs for the edge DNN model (see Fig. 1). Mis-predicted ‘easy’ inputs result in accuracy loss while mis-predicted ‘difficult’ inputs cause unnecessary communication and computation cost. Existing works typically rely on the “confidence” level of the model as the main indicator and design the predictor accordingly [5]–[8]. However, such kind of softmax probability-based confidence measurement has been shown to be inaccurate, as DNNs tend to produce overconfident incorrect predictions [9]–[11].

In this work, we propose to explicitly model the inference difficulty (instead of implicitly estimate a ‘confidence’ level) when training the small edge neural network. To be specific, considering the small edge DNN as an approximator for the large cloud DNN, we develop a two-head architecture for the small edge DNN model that outputs the approximation results and the prediction results simultaneously,

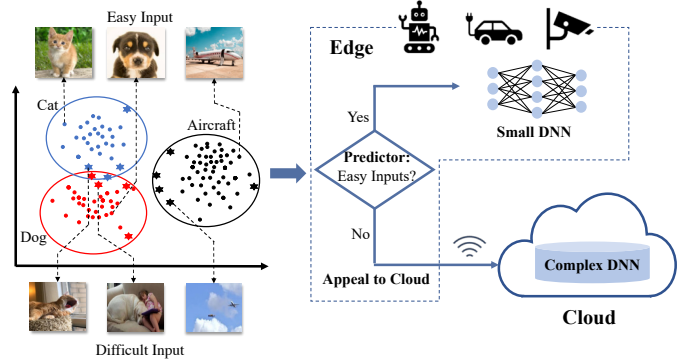


Fig. 1. Edge/Cloud Collaborative Architecture

namely *AppealNet*. The edge/cloud collaborative architecture exploration problem is then formulated as a joint optimization problem to optimize the tradeoff between accuracy and cost, wherein we co-optimize the approximator design and the predictor design on when to appeal to the complex DNN model. We prove that this optimization problem can be solved with a novel training loss representation form, enabling trustworthy differentiation between ‘easy’ and ‘difficult’ inputs. To the best of our knowledge, this is the first work that provides theoretical foundation for edge/cloud collaborative architecture design.

To demonstrate the effectiveness of *AppealNet*, we perform extensive experiments on several image classification tasks using various DNN models. The experimental results show that *AppealNet* is efficient and highly accurate for DNN inference, achieving up to more than 40% energy savings than existing methods, without any accuracy loss.

We organize the remainder of this paper as follows. We review related works in Section II. Section III introduces the problem of edge/cloud collaboration. In Section IV, we present the optimization objective of *AppealNet*. Next, we detail the *AppealNet* architecture and the training method in Section V. Then, Section VI presents our experimental results. Finally, Section VII concludes this paper.

II. RELATED WORK

Applications such as autonomous driving, video surveillance, and speech recognition generate zillions of bytes of data at the network edge, and it is essential to conduct learning to achieve edge intelligence. As edge devices are usually resource-limited, there are both *static* and *dynamic* techniques towards effective deployments of DNNs onto them.

Static techniques include various types of model compression techniques (e.g., *weight pruning* [1], *quantization* [2], *knowledge*

AppealNet: 一种高效、高精度的边缘/云协作 DNN 推理架构

李敏*、李宇*、叶田*、李江[†]和徐强*

*CUhk REliable Computing Laboratory (CURE)、Department of Computer Science and Engineering, The Chinese University of Hong Kong, 香港特别行政区沙田{mli, yuli, tianye, qxu}@cse.cuhk.edu.hk

[†]Department of Computer Science and Engineering, MoE Key Lab of Artificial Intelligence, AI Institute, Shanghai Jiao Tong University, 上海, 中国

Abstract—本文提出了 *AppealNet*, 一种新颖的边缘/云协作架构, 它比最先进的解决方案更有效地运行深度学习 (DL) 任务。对于给定的输入, *AppealNet* *accurately* 会实时预测部署在资源受限的边缘设备上的深度学习模型是否可以成功处理该输入, 如果不能, 则诉诸部署在云端的更强大的深度学习模型。这是通过采用双头神经网络架构来实现的, 该架构明确考虑推理难度, 并优化边缘/云协作架构的准确性和计算/通信成本之间的权衡。多个图像分类数据集的实验结果显示, 与现有技术相比, 在不牺牲准确性的情况下, 节能高达 40% 以上。

一、简介

多年来, 通过采用深度神经网络 (DNN) 模型进行基于学习的处理, 物联网 (IoT) 变得越来越智能。由于此类边缘设备的严格资源限制, 其中使用的 DNN 模型必须很小。为了解决这个问题, 文献中提出了用于边缘计算的各种 DNN 模型压缩技术 (e.g., *weight pruning* [1], *quantization* [2], *knowledge distillation* [3]) 和紧凑的 DNN 架构 [4]。

由于模型容量有限, 小型 DNN 模型通常不如大型 DNN 模型准确。一般来说, 小模型很难处理数据分布长尾中的那些极端情况, 但它们仍然可以正确推断大多数其他输入。因此, 可以采用边缘/云协作架构, 在边缘使用小型 DNN 模型处理这些“简单”输入, 并将“困难”输入转移到云端强大的 DNN 模型 [5]-[8]。与仅边缘或仅云解决方案相比, 这种架构有助于实现 DNN 推理的良好能量精度权衡。

毫无疑问, 这种边缘/云协作架构的有效性取决于负责区分边缘 DNN 模型“简单”和“困难”输入的 *predictor* 的准确性 (见图 1)。错误预测的“简单”输入会导致准确性损失, 而错误预测的“困难”输入会导致不必要的通信和计算成本。现有的工作通常依赖于模型的“置信度”水平作为主要指标, 并相应地设计预测器 [5]-[8]。然而, 这种基于 softmax 概率的置信度测量已被证明是不准确的, 因为 DNN 往往会产生过度自信的错误预测 [9]-[11]。

在这项工作中, 我们建议在训练小边缘神经网络时显式地建模推理难度 (而不是隐式估计“置信度”水平)。具体来说, 将小边缘 DNN 视为大型云 DNN 的逼近器, 我们为小边缘 DNN 模型开发了一种双头架构, 可同时输出逼近结果和预测结果,

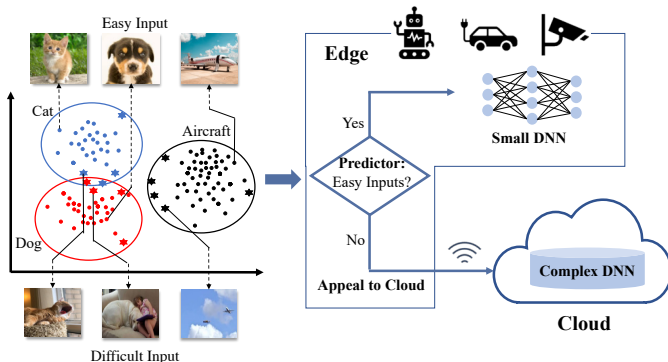


图 1. 边缘/云协作架构

即 *AppealNet*。然后, 边缘/云协作架构探索问题被公式化为联合优化问题, 以优化准确性和成本之间的权衡, 其中我们共同优化近似器设计和预测器设计, 以确定何时吸引复杂的 DNN 模型。我们证明这个优化问题可以通过一种新颖的训练损失表示形式来解决, 从而能够可靠地区分“简单”和“困难”输入。据我们所知, 这是第一篇为边缘/云协作架构设计提供理论基础的工作。

为了证明 *AppealNet* 的有效性, 我们使用各种 DNN 模型对多个图像分类任务进行了广泛的实验。实验结果表明, *AppealNet* 对于 DNN 推理来说是高效且高精度的, 比现有方法节省了 40% 以上的能量, 并且没有任何精度损失。

我们将本文的其余部分组织如下。我们在第二部分回顾相关工作。第三节介绍了边缘/云协作的问题。在第四节中, 我们提出了 *AppealNet* 的优化目标。接下来, 我们在第五节详细介绍 *AppealNet* 架构和训练方法。然后, 第六节介绍我们的实验结果。最后, 第七节总结了本文。

二. 相关工作

自动驾驶、视频监控、语音识别等应用在网络边缘产生数十亿字节的数据, 进行学习以实现边缘智能至关重要。由于边缘设备通常资源有限, 因此有 *static* 和 *dynamic* 技术可以在其上有效部署 DNN。

静态技术包括各种类型的模型压缩技术 (e.g., 权重剪枝 [1], 量化 [2], 知识

distillation [3]) and compact model designs [4]. Given an edge device, they adjust the hyper-parameters of the small edge DNN model (e.g., compression ratio, data types, and channel width) to fit into the device at minimum accuracy degradation from an accurate large model. Dynamic techniques treat incoming data differently and perform selective computations considering that a large portion of inputs do not require a complex DNN model to process.

We can further categorize dynamic DNN solutions into two categories. The first category performs selective computations with model partitioning. For example, [12]–[15] selectively skip some of the computational layers and blocks by inserting extra decision gates in the model. BranchyNet [16], dynamic routing [17], and shallow-deep network [18] early-exit the computation when certain classification confidence from layer-wise outputs is achieved. Moreover, adjusting the network depth according to the input scale provides an alternative way for dynamic processing [19]. While partitioning the complex DNN model across the edge device and cloud does reduce computational cost, it may incur significant communication overheads as the intermediate outputs of complex DNNs could be several orders of magnitude larger than the original raw input [20].

The second category employs multiple DNN models in the system and dynamically selects an appropriate one for processing each input at runtime [6]–[8], [21]. Park *et al.* [6] proposes a Big/Little neural network design for edge/cloud collaboration. They use the difference between the largest and the second-largest softmax confidence value as the indicator to choose which model to use. Later, [7], [8], [21] extend it to a series of models and optimize the collaborative architecture for energy savings.

The Big/Little architecture proposed in [6] is attractive, but using softmax probability-based confidence for model selection is questionable. Several works (e.g., [9], [10]) have shown that the predictive softmax-based distribution from DNNs has a poor relationship with the actual inference confidence. For example, high-confidence prediction are frequently produced for many mis-classified inputs. Therefore, it is essential to develop new indicators to differentiate those ‘easy’ inputs and ‘difficult’ inputs for edge/cloud collaboration.

III. EDGE/CLOUD COLLABORATION PROBLEM

Let us consider a robot vacuum cleaner that is equipped with an image classifier for obstacle avoidance as the edge device. Let input image $\mathbf{x} \in \mathcal{X}$ be the data collected from the cleaner’s camera, and its corresponding classification label $y \in \mathcal{Y}$ (e.g., pets, chairs, or tables) be a random variable that follows a joint data distribution $P(\mathbf{x}, y)$ (the joint distribution of images and the corresponding labels from a specific house), in which $\mathcal{Y} = \{1, \dots, K\}$, K refer to the number of classes, and y is the ground-truth label for input $\mathbf{x}$.

Since the inputs are collected in a dynamic and non-ideal condition, the objects in the images do not always behave in the predictable way. For instance, cats may show up in strange poses (e.g. yawning in Fig. 1), which are beyond the prediction capability of the small model equipped locally in the robot. Given this, if we allow the edge devices to make all predictions locally, the accuracy might be not satisfactory, and further robot actions based on the wrong predictions may have some negative impact. To achieve higher accuracy, an edge/cloud collaborative architecture can be used.

To form such a collaborative architecture, we assume two discriminant models (e.g., convolutional neural networks) are trained on a dataset drawn *i.i.d* from $P(\mathbf{x}, y)$. One is a computationally-intensive neural network using lots of resources at Cloud to achieve high performance, denoted by $f_0 : \mathcal{X} \rightarrow \mathcal{Y}$. The other is a small

edge neural network (i.e., an *approximator* of the large model), with limited capacity and lower accuracy, denoted by $f_1 : \mathcal{X} \rightarrow \mathcal{Y}$.

During the inference stage, a *soft* prediction function (i.e., *predictor*) $q(z|\mathbf{x}) \in [0, 1]$ with $z \in \{0, 1\}$, which is also deployed on the robot, is used to decide whether the small DNN f_1 is adequate to classify the input $\mathbf{x}$ (e.g., common objects in the house without much variation), or the input should be off-loaded to the powerful Cloud DNN model for processing f_0 . To be specific, we adopt the *threshold-based* approach, and assign label 1 to z if $q(z|\mathbf{x})$ is above some threshold δ (e.g., 0.5), and label it 0, otherwise. We expect the predictor to be sufficiently accurate so that only those ‘difficult’ inputs are off-loaded to the Cloud.

Thus, an edge/cloud collaborative architecture is a combination of (f_0, f_1, q) , and the final output of edge/cloud collaborative architecture with respect to a specific input $\mathbf{x}$ is:

$$(f_0, f_1, q)(\mathbf{x}) = \begin{cases} f_1(\mathbf{x}) & \text{if } q(1|\mathbf{x}) \geq \delta, \\ f_0(\mathbf{x}) & \text{otherwise.} \end{cases} \quad (1)$$

The performance of a stand-alone model f_z could be modeled by $\ell(f_z(\mathbf{x}), y)$, $z \in \{0, 1\}$, where $\ell : \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}^+$ is a task loss function, for example, *cross-entropy* loss which is often used in classification tasks. Therefore, taking the respective loss of the complex and small DNN into consideration, the overall expected loss of the edge/cloud collaborative architecture can be calculated by Eq. (2).

$$\begin{aligned} & \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] = \\ & \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x}) \cdot \ell(f_1(\mathbf{x}), y) + (1 - q(1|\mathbf{x})) \cdot \ell(f_0(\mathbf{x}), y)] \end{aligned} \quad (2)$$

$$\begin{aligned} & \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\text{cost}(f_z, q, \mathbf{x})] = \\ & \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x}) \cdot \text{cost}(f_1, q, \mathbf{x}) + (1 - q(1|\mathbf{x})) \cdot \text{cost}(f_0, q, \mathbf{x})] \end{aligned} \quad (3)$$

Eq. (3) shows the cost of the edge/cloud collaboration system. For a given input $\mathbf{x}$, the cost of using a particular model for prediction is denoted by $\text{cost}(f_z, q, \mathbf{x})$ (to be detailed in Section IV-A)

In most edge applications, especially for resource-limited IoT devices, a budget constraint must be imposed on the system. In the edge/cloud collaboration setting, thus, an optimal design can be defined as minimizing the overall expected loss, under a particular cost constraint b . We formally write the optimization objective as follows:

$$\begin{aligned} & \min_{f_0, f_1 \in \mathcal{F}, q \in \mathcal{Q}} \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] \\ & \text{s.t. } \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\text{cost}(f_z, q, \mathbf{x})] \leq b \end{aligned} \quad (4)$$

Note that, the key difference between our formulation and previous work [6]–[8] is that, the predictor function is parameterized and can be co-optimized when training the edge/cloud collaborative models.

IV. PROPOSED RELAXATION METHOD

In this section, we relax the optimization objective of edge/cloud collaboration to make it tractable to solve. We first consider the case that the complex network is a pre-trained white-box model, and separate it from the optimization objective. Then, we look into the cost constraints of (f_0, f_1, q) and simplify them as a single constraint with respect to the expectation of $q(z|\mathbf{x})$. Next, we consider the black-box scenario where the complex DNN is a black box during optimization and present how to relax the optimization objective in this case. Note that, without loss of generality, we consider the optimization objective of classification tasks with cross-entropy loss in the following discussion.

蒸馏[3])和紧凑模型设计[4]。给定边缘设备,他们调整小边缘DNN模型的超参数(e.g.、压缩比、数据类型和通道宽度),以将精确的大型模型的精度降幅降至最低,以适应设备。考虑到大部分输入不需要复杂的DNN模型来处理,动态技术以不同的方式处理输入数据并执行选择性计算。

我们可以进一步将动态DNN解决方案分为两类。第一类通过模型分区执行选择性计算。例如,[12]-[15]通过在模型中插入额外的决策门来选择性地跳过一些计算层和块。BranchyNet [16]、动态路由[17]和浅深网络[18]在逐层输出达到一定的分类置信度时提前退出计算。此外,根据输入尺度调整网络深度为动态处理提供了另一种方式[19]。虽然跨边缘设备和云划分复杂的DNN模型确实降低了计算成本,但它可能会产生大量的通信开销,因为复杂DNN的中间输出可能比原始原始输入大几个数量级[20]。

第二类在系统中采用多个DNN模型,并动态选择一个合适的模型来在运行时处理每个输入[6]-[8]、[21]。Park *et. al* [6]提出了一种用于边缘/云协作的Big/Little神经网络设计。他们使用最大和第二大softmax置信值之间的差异作为选择使用哪个模型的指标。后来,[7]、[8]、[21]将其扩展到一系列模型并优化协作架构以实现节能。

[6]中提出的Big/Little架构很有吸引力,但使用基于softmax概率的置信度进行模型选择是有问题的。一些研究(e.g.、[9]、[10])表明,DNN中基于softmax的预测分布与实际推理置信度的关系较差。例如,对于许多错误分类的输入经常会产生高置信度预测。因此,有必要开发新的指标来区分边缘/云协作的“简单”输入和“困难”输入。

三. 边缘/云协作问题

让我们考虑一个机器人吸尘器,它配备了用于避障的图像分类器作为边缘设备。设输入图像 $\mathbf{x} \in \mathcal{X}$ 是从清洁工的摄像头收集的数据,其对应的分类标签 $y \in \mathcal{Y}$ (e.g.、宠物、椅子或桌子)是遵循联合数据分布的随机变量 $P(\mathbf{x}, y)$ (特定房屋的图像和相应标签的联合分布),其中 $\mathcal{Y} = \{1, \dots, K\}$ 、 K 指类别数, y 是类别数输入 $\mathbf{x}$ 的真实标签。

由于输入是在动态和非理想条件下收集的,因此图像中的对象并不总是以可预测的方式表现。例如,猫可能会以奇怪的姿势出现(例如图中打哈欠),这超出了机器人本地配备的小模型的预测能力。鉴于此,如果我们允许边缘设备在本地进行所有预测,则准确性可能无法令人满意,并且基于错误预测的进一步机器人动作可能会产生一些负面影响。为了实现更高的精度,可以使用边缘/云协作架构。

为了形成这样的协作架构,我们假设在从 $P(\mathbf{x}, y)$ 提取的*i.i.d*数据集上训练两个判别模型(e.g., 卷积神经网络)。一种是计算密集型神经网络,使用云中的大量资源来实现高性能,用 $f_0: \mathcal{X} \rightarrow \mathcal{Y}$ 表示。另一个是小号

边缘神经网络(i.e., 大模型的*approximator*),容量有限,精度较低,记为 $f_1: \mathcal{X} \rightarrow \mathcal{Y}$ 。

在推理阶段,使用同样部署在机器人上的*soft*预测函数(i.e., *pre-dictor*) $q(z|\mathbf{x}) \in [0, 1]$ 和 $z \in \{0, 1\}$ 来确定小型DNN f_1 是否足以对输入 $\mathbf{x}$ (e.g.、房子里没有太多变化的常见物体)进行分类,或者输入应卸载到强大的Cloud DNN模型来处理 f_0 。具体来说,我们采用*threshold-based*方法,如果 $q(z|\mathbf{x})$ 高于某个阈值 δ (e.g.、0.5),则将标签1分配给 z ,否则将其标记为0。我们期望预测器足够准确,以便只有那些“困难”的输入才会被卸载到云端。

因此,边缘/云协作架构是 (f_0, f_1, q) 的组合,并且边缘/云协作架构相对于特定输入 $\mathbf{x}$ 的最终输出是:

$$(f_0, f_1, q)(\mathbf{x}) = \begin{cases} f_1(\mathbf{x}) & \text{if } q(1|\mathbf{x}) \geq \delta, \\ f_0(\mathbf{x}) & \text{otherwise.} \end{cases} \quad (1)$$

独立模型 f_z 的性能可以通过 $\ell(f_z(\mathbf{x}), y), z \in \{0, 1\}$ 建模,其中 $\ell: \mathcal{X} \times \mathcal{Y} \rightarrow \mathbb{R}^+$ 是任务损失函数,例如,分类任务中经常使用的*cross-entropy*损失。因此,考虑到复杂和小型DNN各自的损失,边缘/云协作架构的总体预期损失可以通过式(1)计算:(2)。

$$\begin{aligned} & \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] = \\ & \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x}) \cdot \ell(f_1(\mathbf{x}), y) + (1 - q(1|\mathbf{x})) \cdot \ell(f_0(\mathbf{x}), y)] \quad (2) \end{aligned}$$

$\mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\text{cost}(f_z, q, \mathbf{x})] = \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x}) \cdot \text{cost}(f_1, q, \mathbf{x}) + (1 - q(1|\mathbf{x})) \cdot \text{cost}(f_0, q, \mathbf{x})] \quad (3)$ 等式。(3)显示了边缘/云协作系统的成本。对于给定的输入 $\mathbf{x}$,使用特定模型进行预测的成本由 $\text{cost}(f_z, q, \mathbf{x})$ (表示,将在第IV-A节中详细介绍)

在大多数边缘应用中,特别是对于资源有限的物联网设备,必须对系统施加预算约束。因此,在边缘/云协作设置中,最佳设计可以定义为在特定成本约束 b 下最小化总体预期损失。我们将优化目标写如下:

$$\begin{aligned} & \min_{f_0, f_1 \in \mathcal{F}, q \in \mathbf{Q}} \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] \\ & \text{s.t. } \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\text{cost}(f_z, q, \mathbf{x})] \leq b \quad (4) \end{aligned}$$

请注意,我们的公式与之前的工作[6]-[8]之间的主要区别在于,预测函数是参数化的,并且可以在训练边缘/云协作模型时共同优化。

四. 建议的放松方法

在本节中,我们放宽了边缘/云协作的优化目标,使其易于解决。我们首先考虑复杂网络是预先训练的白盒模型的情况,并将其与优化目标分开。然后,我们研究 (f_0, f_1, q) 的成本约束并将其简化为相对于 $q(z|\mathbf{x})$ 的期望的单个约束。接下来,我们考虑黑盒场景,其中复杂的DNN在优化过程中是一个黑盒,并介绍在这种情况下如何放松优化目标。请注意,在不失一般性的情况下,我们在下面的讨论中考虑具有交叉熵损失分类任务的优化目标。

A. Objective Relaxation

As shown in Eq. (4), our objective is to minimize the expected overall loss of the edge/cloud collaborative DNN models, under a given constraint on the total system cost. The output of the optimization procedure is the combination of (f_0, f_1, g) .

The main challenge for solving this optimization problem is its huge solution space. In practice, the complex DNN could be an ensemble of various models or a very deep network to achieve state-of-the-art performance. Consequently, optimizing f_0 and f_1 together may lead to very slow convergence. To reduce complexity, we assume a known fixed complex DNN f_0 with high accuracy. Note that, the fixed complex DNN is available when training the small edge DNN so that we could get the loss item $\ell(f_0(\mathbf{x}), y)$ for optimization.

Cost constraints. As shown in Eq. (3), with the predictor function q , the cost to infer an input $\mathbf{x}$ depends on which DNN to execute it. For a static DNN, the cost for its inference is independent of the specific input x . Therefore, we have $\text{cost}(f_z, q, \mathbf{x}) = \text{cost}(f_z, q)$.

Let us consider the standalone DNN model first. The main cost include off-chip memory access of DNN parameters and neuron computation. As both are directly related to the computational demands, we can simply use computational cost (in unit of FLOPS) to approximate them.

With edge/cloud collaboration, some inputs are off-loaded to the cloud and the results are then sent back to the edge device. Thus, running the predictor at the edge and the communication between the edge and the cloud also contributes to the total cost. We can use a constant $c_1 = \text{cost}(f_1, q)$ to represent the cost of running predictor/small DNN on the edge device and another constant $c_0 = \text{cost}(f_0, q)$ to represent the accumulated cost of running predictor on the edge device, running complex DNN on the cloud, and the communication overhead.

Often the related cost of processing the input by the complex DNN on the cloud is much larger than that by the small edge DNN. Therefore, we have use $c_0 \gg c_1$. By substituting Eq. (3) with c_0 and c_1 , we have the reduced cost function as Eq. (5).

$$(c_1 - c_0) \cdot \mathbb{E}_{P(\mathbf{x}, y)}[q(1|\mathbf{x})] + c_0 \quad (5)$$

As the cost is under a specific budget b ($c_1 < b < c_0$), we can derive the simplified constraint as Eq. (6).

$$\mathbb{E}_{P(\mathbf{x}, y)}[q(1|\mathbf{x})] \geq \hat{b}, \hat{b} = \frac{c_0 - b}{c_0 - c_1} \quad (6)$$

According to Eq. (6), $q(1|\mathbf{x})$ is an indicator of the overall cost, and the new cost constraint $\hat{b} \in (0, 1)$ ensures at least $\hat{b}$ fraction of inputs would be processed by the small edge DNN.

Relaxed optimization objective. With the above, we have the constrained optimization objective of edge/cloud collaborative design as follows:

$$\begin{aligned} \min_{f_1 \in \mathcal{F}, q \in \mathcal{Q}} \quad & \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] \\ \text{s.t.} \quad & \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x})] \geq \hat{b} \end{aligned} \quad (7)$$

To convert the constraint into objective itself, we further take $\log(\cdot)$ of predictor likelihood $q(z|\mathbf{x})$, so that the constraint becomes:

$$\log(\hat{b}) - \mathbb{E}_{P(\mathbf{x}, y)} [\log(q(1|\mathbf{x}))] \leq 0 \quad (8)$$

Re-writing Eq. (7) as a *Lagrangian* under *KKT* conditions [22], we can put the constraint (8) into the objective function, remove the

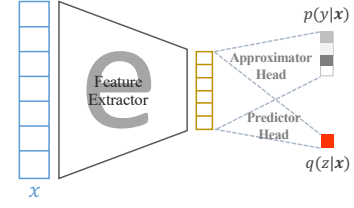


Fig. 2. Two-head architecture for the little network in AppealNet.

constant term $\log(\hat{b})$, and obtain the final optimization objective as follows:

$$\min_{f_1, q} \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] + \beta \cdot \mathbb{E}_{P(\mathbf{x}, y)} [-\log(q(1|\mathbf{x}))] \quad (9)$$

B. Black-Box Approximation

In the earlier discussion, we assume the complex DNN is a white-box neural network, where its output information is available so that we could calculate the classification loss $\ell(f_0(\mathbf{x}), y)$ for a specific input. However, the complex DNN might be unknown during optimization, in many cases. For example, in our edge-cloud collaboration scenarios, the complex DNN might be provided by a machine learning service vendor running at a remote data center, and it is agnostic to the small DNN to be deployed on the edge.

Under such circumstances, we propose to treat the black-box complex DNN as an oracle function $\mathcal{O}(\cdot)$, i.e., it can correctly classify every input from $P(\mathbf{x}, y)$. Now, the loss term $\ell(f_0(\mathbf{x}), y) = \ell(\mathcal{O}(\mathbf{x}), y)$ in Eq. (9) becomes zero as the outputs of the oracle function are always the ground-truth values. Therefore, the optimization objective becomes:

$$\min_{f_1, q} \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x}) \cdot \ell(f_1(\mathbf{x}), y)] + \beta \cdot \mathbb{E}_{P(\mathbf{x}, y)} [-\log(q(1|\mathbf{x}))] \quad (10)$$

V. APPEALNET DESIGN

In this section, we present the overall architecture of AppealNet and its training method, in which we jointly train the little network and the predictor in an end-to-end manner. We also illustrate the full workflow for AppealNet.

A. Two-Head Architecture

Fig. 2 demonstrates the two-head design architecture, which consists of one common feature extractor and two separate heads (i.e., *approximator head* and *predictor head*). Using a two-head design has several benefits. First, merging $\mathcal{F}$ and $\mathcal{Q}$ into one unified design space enables us to utilize and modify *off-the-shelf* efficient DNN architectures (e.g., EfficientNet [23], ShuffleNet [24], MobileNet [4]) for the approximator/predictor design. For a given device with specific resource constraints, we can first select an efficient DNN architecture that meets this constraint and add a predictor head to the original architecture with minimal overhead. Second, such a design facilitates training (f_0, q) jointly in an end-to-end manner.

To be more specific, we parameterize the predictor function $q(z|\mathbf{x})$ as a neural network, and enforce f_1 and q sharing all computational-intensive layers (e.g., convolutional layers) wrapped in the *feature extractor*. Thus, the approximator consists of the feature extractor and an approximator head, while the predictor consists of the feature extractor and a predictor head. For classification tasks, the approximator head contains several cascaded fully-connected layers and a *softmax* layer to output the predictive probability vector $p(y|\mathbf{x})$, wherein the i^{th} element indicates the probability of the i^{th} class. For the predictor head used in AppealNet, we simply employ a single fully-connected layer to keep the design overhead minimum. The output of predictor passes through a *sigmoid* activation function such that $q(z|\mathbf{x}) \in [0, 1]$.

A. Objective Relaxation

如方程式所示。(4)，我们的目标是在给定的总系统成本约束下，最小化边缘/云协作 DNN 模型的预期总体损失。优化过程的输出是 (f_0, f_1, q) 的组合。

解决这个优化问题的主要挑战是其巨大的解空间。在实践中，复杂的 DNN 可以是各种模型的集合或非常深的网络，以实现最先进的性能。因此，同时优化 f_0 和 f_1 可能会导致收敛速度非常慢。为了降低复杂性，我们假设一个已知的固定复杂 DNN f_0 具有高精度。请注意，在训练小边缘 DNN 时，可以使用固定复杂 DNN，以便我们可以得到损失项 $\ell(f_0(\mathbf{x}), y)$ 进行优化。

成本限制。如方程式所示。(3)，使用预测函数 q ，推断输入 $\mathbf{x}$ 的成本取决于执行哪个 DNN。对于静态 DNN，其推理成本与特定输入 x 无关。因此，我们有 $\text{cost}(f_z, q, \mathbf{x}) = \text{cost}(f_z, q)$ 。

让我们首先考虑独立的 DNN 模型。主要成本包括 DNN 参数的片外内存访问和神经元计算。由于两者都与计算需求直接相关，因此我们可以简单地使用计算成本（以 FLOPS 为单位）来近似它们。

通过边缘/云协作，一些输入被卸载到云端，然后结果被发送回边缘设备。因此，在边缘运行预测器以及边缘和云之间的通信也会增加总成本。我们可以使用常量 $c_1 = \text{cost}(f_1, q)$ 来表示在边缘设备上运行预测器/小型 DNN 的成本，并使用另一个常量 $c_0 = \text{cost}(f_0, q)$ 来表示在边缘设备上运行预测器、在云上运行复杂 DNN 的累积成本以及通信开销。

通常，云上复杂 DNN 处理输入的相关成本远大于小型边缘 DNN。因此，我们使用 $c_0 \gg c_1$ 。通过代入方程。(3) 对于 c_0 和 c_1 ，我们可以得到如下的降低成本函数：(5)。

$$(c_1 - c_0) \cdot \mathbb{E}_{P(\mathbf{x}, y)}[q(1|\mathbf{x})] + c_0 \quad (5)$$

由于成本处于特定预算 b ($c_1 < b < c_0$) 之下，我们可以推导出简化的约束，如式 (1) 所示。(6)。

$$\mathbb{E}_{P(\mathbf{x}, y)}[q(1|\mathbf{x})] \geq \hat{b}, \hat{b} = \frac{c_0 - b}{c_0 - c_1} \quad (6)$$

根据方程。(6)， $q(1|\mathbf{x})$ 是总体成本的指标，新的成本约束 $\hat{b} \in (0, 1)$ 确保至少 $\hat{b}$ 部分输入将由小边缘 DNN 处理。

宽松的优化目标。综上所述，我们得到边云协同设计的约束优化目标如下：

$$\begin{aligned} \min_{f_1 \in \mathcal{F}, q \in \mathcal{Q}} \quad & \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] \\ \text{s.t.} \quad & \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x})] \geq \hat{b} \end{aligned} \quad (7)$$

为了将约束转换为目标本身，我们进一步采用预测器似然 $q(z|\mathbf{x})$ 的 $\log()$ ，使得约束变为：

$$\log(\hat{b}) - \mathbb{E}_{P(\mathbf{x}, y)} [\log(q(1|\mathbf{x}))] \leq 0 \quad (8)$$

重写方程。(7) 作为 KKT 条件下的 Lagrangian [22]，我们可以将约束(8)代入目标函数，去掉

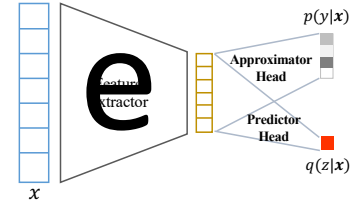


图 2. AppealNet 中小型网络的双头架构。常数项 $\log(\hat{b})$ ，得到最终的优化目标如下：

$$\min_{f_1, q} \mathbb{E}_{P(\mathbf{x}, y)} \mathbb{E}_{q(z|\mathbf{x})} [\ell(f_z(\mathbf{x}), y)] + \beta \cdot \mathbb{E}_{P(\mathbf{x}, y)} [-\log(q(1|\mathbf{x}))] \quad (9)$$

B. Black-Box Approximation

在前面的讨论中，我们假设复杂的 DNN 是一个白盒神经网络，其输出信息可用，以便我们可以计算特定输入的分类损失 $\ell(f_0(\mathbf{x}), y)$ 。然而，在许多情况下，复杂的 DNN 在优化过程中可能是未知的。例如，在我们的边缘-云协作场景中，复杂的 DNN 可能由在远程数据中心运行的机器学习服务供应商提供，并且与部署在边缘上的小型 DNN 无关。

在这种情况下，我们建议将黑盒复杂 DNN 视为预言机函数 $\mathcal{O}(\cdot)$ ，i.e.，它可以正确地对来自 $P(\mathbf{x}, y)$ 的每个输入进行分类。现在，方程中的损失项 $\ell(f_0(\mathbf{x}), y) = \ell(\mathcal{O}(\mathbf{x}), y)$ (9) 变为零，因为预言函数的输出始终是真实值。因此，优化目标变为：

$$\min_{f_1, q} \mathbb{E}_{P(\mathbf{x}, y)} [q(1|\mathbf{x}) \cdot \ell(f_1(\mathbf{x}), y)] + \beta \cdot \mathbb{E}_{P(\mathbf{x}, y)} [-\log(q(1|\mathbf{x}))] \quad (10)$$

五、申诉网设计

在本节中，我们介绍 AppealNet 的整体架构及其训练方法，其中我们以端到端的方式联合训练小网络和预测器。我们还说明了 AppealNet 的完整工作流程。

A. Two-Head Architecture

图2展示了双头设计架构，它由一个公共特征提取器和两个独立的头 (i.e.、approximator head 和 predictor head) 组成。使用双头设计有几个好处。首先，将 $\mathcal{F}$ 和 $\mathcal{Q}$ 合并到一个统一的设计空间中，使我们能够利用和修改 off-the-shelf 高效 DNN 架构 (e.g.、EfficientNet [23]、ShuffleNet [24]、MobileNet [4]) 进行逼近器/预测器设计。对于具有特定资源约束的给定设备，我们可以首先选择满足此约束的高效 DNN 架构，并以最小的开销向原始架构添加预测器头。其次，这样的设计有利于以端到端的方式联合训练 (f_0, q) 。

更具体地说，我们将预测函数 $q(z|\mathbf{x})$ 参数化为神经网络，并强制 f_1 和 q 共享包含在 feature extractor 中的所有计算密集型层 (e.g.、卷积层)。因此，逼近器由特征提取器和逼近器头组成，而预测器由特征提取器和预测器头组成。对于分类任务，逼近器头包含几个级联的全连接层和一个 softmax 层，用于输出预测概率向量 $p(y|\mathbf{x})$ ，其中 i^{th} 元素表示 i^{th} 类的概率。对于 AppealNet 中使用的预测器头，我们只需采用单个全连接层即可将设计开销保持在最低限度。预测器的输出通过 sigmoid 激活函数，使得 $q(z|\mathbf{x}) \in [0, 1]$ 。

Algorithm 1: Joint Training for Two-Head Small Network.

Input: Dataset $\mathbf{S}$ sampled from $P(\mathbf{x}, y)$, Big Network f_0
Output: Little Network (f_1, q) .
 /* Initialize */
 1 Initialize f_1 with the pre-trained model;
 2 Insert the predictor head and get two-head little network (f_1, q) ;
 /* Define Loss Functions; ℓ depends on tasks and settings */
 3 $\mathcal{L}_p = q(1|\mathbf{x}) \cdot \ell(f_1(\mathbf{x}), y) + (1 - q(1|\mathbf{x})) \cdot \ell(f_0(\mathbf{x}), y)$;
 4 $\mathcal{L}_q = -\log(q(1|\mathbf{x}))$;
 /* Training Process */
 5 **while** Not Convergence **do**
 6 Sample $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_M, y_M)$ from $\mathbf{S}$;
 7 $\mathcal{L} = \frac{1}{M} \sum_i [\mathcal{L}_p(\mathbf{x}_i, y_i | f_0, f_1, q) + \beta \mathcal{L}_q(\mathbf{x}_i | q)]$;
 8 Update (f_1, q) by descending stochastic gradient of $\mathcal{L}$;
 9 **end**
 10 **return** Little Network (f_1, q) ;

B. Joint Training Method

Alg. 1 shows the joint training scheme, which optimizes the approximator f_1 and the predictor q jointly with the help of the big network f_0 . To be specific, we initialize the network parameters using the pre-trained little network without the predictor head first. This step facilitates us to take advantage of available model zoos (e.g., PyTorch model zoo) and accelerate the training process (line 1). Then we insert the predictor head into the original network and start training (line 2). For each batch of training samples, we calculate the loss $\mathcal{L}$, which consists of the system loss and system cost (see line 5). The loss is back-propagated to update the parameters of (f_1, q) . These training steps are repeated until convergence is reached.

C. Full Workflow for AppealNet

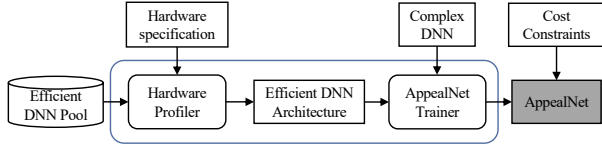


Fig. 3. Working flow of AppealNet

Fig. 3 shows an overview of the overall workflow of AppealNet. First, given the hardware specification and a pool of efficient DNN candidate models, the hardware profiler analyzes which DNN model meets the resource constraints of the targeted hardware platform, and outputs a suitable small edge DNN architecture. Next, the architecture is augmented by the predictor head and fed into the AppealNet trainer for joint training with the help of the complex DNN model, under a given cost constraint with edge/cloud collaboration.

VI. EXPERIMENTAL RESULTS

A. Experimental Setup

Datasets: We demonstrate the effectiveness of AppealNet using four classification datasets: The German Traffic Sign Recognition Benchmark (GTSRB) [25], CIFAR-10, CIFAR-100 [26], and Tiny-ImageNet [27].

Neural Networks: We evaluate AppealNet design by applying it to three off-the-shelf efficient CNN families: MobileNet [4], EfficientNet [23], and ShuffleNet [24]. They can be deployed on a

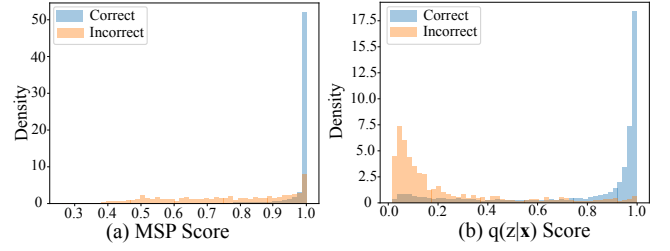


Fig. 4. Histogram of MSP and AppealNet on CIFAR-10.

diverse range of mobile/IoT devices with different hardware resource capacity. We choose ResNet-101 [20] as the complex network.

Baseline Solutions: We consider conventional confidence-score based methods as baseline solutions. These baseline solutions mainly adopt the following score metrics to decide which neural network, the small edge DNN or the complex cloud DNN, to execute:

- 1) Maximum softmax probability (*MSP*) [9] is defined as the maximum probability of a misclassified inputs. *MSP* is obtained from the softmax function: $MSP = 1^{st} p(y|\mathbf{x})$.
- 2) The gap/score margin [6]–[8] (*SM*) is defined as the difference between the largest output probability and the second largest one: $SM = 1^{st} p(y|\mathbf{x}) - 2^{nd} p(y|\mathbf{x})$.
- 3) The entropy score [16] (*Entropy*) is derived by computing the entropy of the output predictive distribution of each input and further taking its minus: $Entropy = \sum_j p(y|\mathbf{x})_j \log[p(y|\mathbf{x})_j]$.

Evaluation Metrics: We define the skipping rate (*SR*) – the fraction of inputs being selected by the predictor to process with the little network – to evaluate the proposed predictor ($q(z|\mathbf{x})$):

$$SR = \frac{1}{N} \sum_i \mathbb{I}(q(1|\mathbf{x}_i) \geq \delta) \quad (11)$$

In contrast, the appealing rate (*AR*) represents the portion of inputs being off-loaded to the big network:

$$AR = \frac{1}{N} \sum_i \mathbb{I}(q(1|\mathbf{x}_i) < \delta) \quad (12)$$

The overall accuracy of the edge/cloud architecture for DNN inference can be calculated as follows:

$$\begin{aligned} \text{Acc.}(f_0, f_1, g) = \frac{1}{N} \sum_i [\mathbb{I}(q(1|\mathbf{x}_i) \geq \delta) \mathbb{I}(f_1(\mathbf{x}_i) = y_i) \\ + \mathbb{I}(q(1|\mathbf{x}_i) < \delta) \mathbb{I}(f_0(\mathbf{x}_i) = y_i)] \end{aligned} \quad (13)$$

We define the relative accuracy improvement (*AccI*) as the improvement of classification accuracy of the edge/cloud collaborative architecture compared with the stand-alone small DNN deployed on the edges, scaled by the accuracy gap between two DNNs.

$$\text{AccI} = \frac{\text{Acc.}(f_0, f_1, q) - \text{Acc.}(f_1)}{\text{Acc.}(f_0) - \text{Acc.}(f_1)} \quad (14)$$

We use the computational cost of the overall edge/cloud system for cost comparison:

$$\text{Cost}_{\text{overall}} = SR \times \text{cost}(f_1, q) + (1 - SR) \times \text{cost}(f_0, q) \quad (15)$$

Consider cloud-edge collaborative architectures, the bandwidth and latency cost of off-loading the data from edge to cloud are directly related to the skipping rate of inputs being processed by the small network on the edge; while the performance and energy cost of the whole system are evaluated by the overall accuracy improvement and cost listed above.

Algorithm 1: Joint Training for Two-Head Small Network.

Input: Dataset $\mathbf{S}$ sampled from $P(\mathbf{x}, y)$, Big Network f_0

Output: Little Network (f_1, q) .

/* Initialize */

1 Initialize f_1 with the pre-trained model;

2 Insert the predictor head and get two-head little network (f_1, q) ;

/* Define Loss Functions; ℓ depends on tasks and settings */

3 $\mathcal{L}_p = q(1|\mathbf{x}) \cdot \ell(f_1(\mathbf{x}), y) + (1 - q(1|\mathbf{x})) \cdot \ell(f_0(\mathbf{x}), y)$;

4 $\mathcal{L}_q = -\log(q(1|\mathbf{x}))$;

/* Training Process */

5 **while** Not Convergence **do**

6 Sample $(\mathbf{x}_1, y_1), \dots, (\mathbf{x}_M, y_M)$ from $\mathbf{S}$;

7 $\mathcal{L} = \frac{1}{M} \sum_i [\mathcal{L}_p(\mathbf{x}_i, y_i | f_0, f_1, q) + \beta \mathcal{L}_q(\mathbf{x}_i | q)]$;

8 Update (f_1, q) by descending stochastic gradient of $\mathcal{L}$;

9 **end**

10 **return** Little Network (f_1, q) ;

B. Joint Training Method

藻类。图 1 显示了联合训练方案，该方案在大网络 f_0 的帮助下联合优化逼近器 f_1 和预测器 q 。具体来说，我们首先使用预先训练的小网络（没有预测器头）来初始化网络参数。此步骤有助于我们利用可用的模型动物园（e.g., PyTorch 模型动物园）并加速训练过程（第 1 行）。然后我们将预测器头插入原始网络并开始训练（第 2 行）。对于每一批训练样本，我们计算损失 $\mathcal{L}$ ，它由系统损失和系统成本组成（见第 5 行）。损失被反向传播以更新 (f_1, q) 的参数。重复这些训练步骤直到达到收敛。

C. Full Workflow for AppealNet

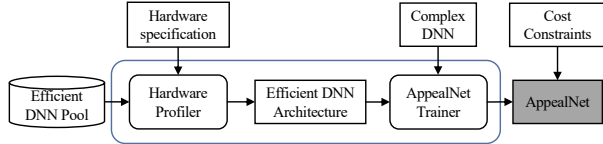


图3. AppealNet的工作流程

图 3 显示了 AppealNet 整体工作流程的概述。首先，给定硬件规格和一组高效的 DNN 候选模型，硬件分析器分析哪个 DNN 模型满足目标硬件平台的资源限制，并输出合适的小边缘 DNN 架构。接下来，该架构由预测器头进行增强，并在复杂的 DNN 模型的帮助下，在给定的边缘/云协作成本约束下，输入 Appeal Net 训练器进行联合训练。

六. 实验结果

A. Experimental Setup

数据集：我们使用四个分类数据集证明了 AppealNet 的有效性：德国交通标志识别基准 (GTSRB) [25]、CIFAR-10、CIFAR-100 [26] 和 TinyImageNet [27]。

神经网络：我们通过将 AppealNet 设计应用于三个现成的高效 CNN 系列来对其进行评估：MobileNet [4]、EfficientNet [23] 和 ShuffleNet [24]。它们可以部署在

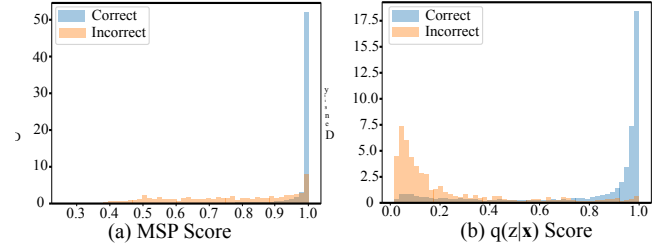


图 4. CIFAR-10 上 MSP 和 AppealNet 的直方图。

具有不同硬件资源容量的各种移动/物联网设备。我们选择 ResNet-101 [20] 作为复杂网络。

基线解决方案：我们将传统的基于置信度评分的方法视为基线解决方案。这些基线解决方案主要采用以下评分指标来决定执行哪种神经网络、小型边缘 DNN 还是复杂的云 DNN：

- 1) 最大softmax概率 (MSP) [9]被定义为错误分类输入的最大概率。MSP由softmax函数得到： $MSP = 1^{st} p(y|\mathbf{x})$ 。2) 差距/分数差距[6]–[8] (SM)被定义为最大输出概率和第二大输出概率之间的差： $SM = 1^{st} p(y|\mathbf{x}) - 2^{nd} p(y|\mathbf{x})$ 。3) 熵得分[16] (Entropy) 是通过计算每个输入的预测分布的熵并进一步取其负值： $Entropy = -\sum_j p(y|\mathbf{x})_j \log[p(y|\mathbf{x})_j]$ 得出的。

评估指标：我们定义跳跃率 (SR) – 预测器选择使用小网络进行处理的输入的比例 – 来评估建议的预测器 ($q(z|\mathbf{x})$):

$$SR = \frac{1}{N} \sum_i \mathbb{I}(q(1|\mathbf{x}_i) \geq \delta) \quad (11)$$

相反，吸引率 (AR) 代表被卸载到大网络的输入部分：

$$AR = \frac{1}{N} \sum_i \mathbb{I}(q(1|\mathbf{x}_i) < \delta) \quad (12)$$

DNN 推理的边缘/云架构的整体精度可以计算如下：

$$Acc.(f_0, f_1, g) = \frac{1}{N} \sum_i [\mathbb{I}(q(1|\mathbf{x}_i) \geq \delta) \mathbb{I}(f_1(\mathbf{x}_i) = y_i) + \mathbb{I}(q(1|\mathbf{x}_i) < \delta) \mathbb{I}(f_0(\mathbf{x}_i) = y_i)] \quad (13)$$

我们将相对精度改进 (AccI) 定义为与部署在边缘的独立小型 DNN 相比，边缘/云协作架构的分类精度的改进，并按两个 DNN 之间的精度差距进行缩放。

$$AccI = \frac{Acc.(f_0, f_1, q) - Acc.(f_1)}{Acc.(f_0) - Acc.(f_1)} \quad (14)$$

我们使用整体边缘/云系统的计算成本进行成本比较：

$$Cost_{overall} = SR \times cost(f_1, q) + (1 - SR) \times cost(f_0, q) \quad (15)$$

考虑云边协作架构，将数据从边缘卸载到云的带宽和延迟成本与边缘小型网络处理的输入的跳过率直接相关；而整个系统的性能和能源成本则通过上面列出的整体精度提升和成本来评估。

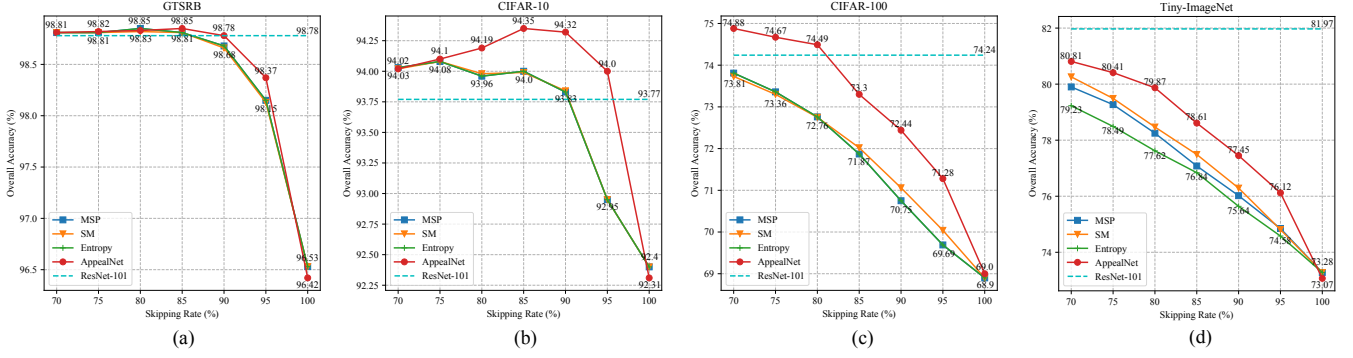


Fig. 5. Overall accuracy under different skipping rate for MobileNet on GTSRB, CIFAR10, CIFAR100 and Tiny-ImageNet.

Dataset	Accuracy %	Cost MFLOPs	Cost at 50.0% AccI	Relative Saving at 50.0% AccI	Cost at 75.0% AccI	Relative Saving at 75.0% AccI	Cost at 90.0% AccI	Relative Saving at 90.0% AccI	Cost at 95.0% AccI	Relative Saving at 95.0% AccI
	ResNet-101/MobileNet/AppealNet		(SM/AppealNet)		(SM/AppealNet)		(SM/AppealNet)		(SM/AppealNet)	
GTSRB	98.78/96.53/96.42	2520.3/28.8/28.8	122.23/100.80	17.53%	192.24/123.48	35.77%	228.37/166.58	27.07%	227.20/191.99	15.50%
CIFAR-10	93.77/92.40/92.31	2520.3/94.6/94.6	217.10/138.02	36.43%	245.72/161.31	34.35%	291.32/164.70	43.46%	314.13/167.86	46.56%
CIFAR-100	74.24/68.90/69.00	2520.4/94.7/94.7	415.38/234.66	43.51%	634.41/385.54	39.22%	819.50/462.19	43.60%	884.51/513.89	41.90%
Tiny-ImageNet	81.97/73.28/73.07	7832.34/313.11/313.11	1482.35/1067.29	28.00%	2215.48/1724.47	22.16%	3089.21/2646.33	14.34%	4374.25/3835.71	12.31%

TABLE I

OVERALL COMPUTATIONAL COST OF EDGE/CLOUD ARCHITECTURE UNDER DIFFERENT ACCURACY REQUIREMENTS.

B. Visualization of Input Differentiation

In Figure 4, we visualize the MSP score and $q(z|x)$ score with EfficientNet as the small edge network and the training dataset is CIFAR-10. The *correct* and *incorrect* inputs refer to those inputs that can be correctly processed at edge and those that cannot be processed, respectively. These two figures clearly show the advantage of AppealNet. That is, AppealNet is able to separate the “easy” inputs and “difficult” inputs well with respect to $q(z|x)$ scores, while MSP scores¹ lead to much more overlap between these two kinds of inputs. Such advantage is obtained with the customized predictor head in the AppealNet architecture and the corresponding training strategy with theoretical foundations. Similar phenomena are also observed for other datasets and models.

C. Effectiveness of AppealNet

In this subsection, we demonstrate the effectiveness of AppealNet in terms of the tradeoff between edge/cloud collaboration system accuracy (defined in Eq. (13)) and energy-saving (e.g. skipping rate defined in Eq. (11) or computational cost defined in Eq. (15)).

First, we show the system accuracy under different skipping rate in Fig. 5². As can be observed from this figure, the accuracy of AppealNet (the red line) is above the baseline methods in most cases across all three datasets. Such results are in line with our expectation as AppealNet can differentiate ‘easy’ and ‘difficult’ inputs much better than baseline solutions. Take Fig. 5 (b) as an example, when the skipping rate is 95% (i.e., 5% of inputs are offloaded to the complex network), the system accuracy is 94.0% with AppealNet while baseline solutions can only achieve 92.95%. We can also see that the relative advantage of AppealNet over baseline solutions (the margin between the red line and other lines) is usually higher with increasing skipping rate (before reaching 100%). This is because, the difference in such differentiation capabilities plays a more important role when the skipping rate is higher.

¹The results of SM scores and Entropy scores are similar to that of MSP scores.

²Here, we only show our results when MobileNet is used as the small edge network. We see similar results with EfficientNet and ShuffleNet.

One might expect that the overall accuracy of the edge/cloud system would be lower than a stand-alone large network since the small network is of low capacity. A seemingly surprising observation from Fig. 5 is that the edge/cloud system design could achieve accuracy boosting. Except for Tiny-ImageNet, the overall accuracy of the edge/cloud system can be above the accuracy of single ResNet-101 (dotted line) in certain skipping rate ranges. This is because, even though the small network has lower accuracy for the entire dataset, it is co-optimized with the big network to minimize the overall expected loss (as shown in Eq. (9)), and it is likely to correctly predict a portion of inputs that cannot be correctly classified by the big network. Compared to other confidence-based approaches, for the more difficult CIFAR-100 dataset, only AppealNet can generate higher accuracy than the standalone large network, as can be seen in Figure 5(c). As for Tiny-ImageNet, the accuracy gap between the large and the small networks are too large (more than eight percent) to enable accuracy boosting when the skipping rate is in the range of [70, 100]. Under such circumstances, we could still benefit from AppealNet that provides a better tradeoff between prediction accuracy and energy savings when compared to other baseline solutions.

As we observe the score margin baseline performs slightly better than the other two baselines in most cases, in later results, we only compare AppealNet with it.

Table I presents the comparison between baseline methods and AppealNet by calculating the overall computational cost (defined in Eq. (15)), measured in MFLOPs, under different accuracy improvement (defined in Eq. (14)). The left part of the table present the accuracy and computational cost of the original ResNet-101, MobileNet and AppealNet, respectively. The key difference between AppealNet and MobileNet is that AppealNet has an extra predictor head and is optimized with a joint training scheme (See Section V-B), and the accuracy of the two-head AppealNet is close to the accuracy of MobileNet (See the second column in Table I).

We now turn to the right part of Table I, which shows the system computational cost with threshold δ tuned to match the target system accuracy. We evaluate four settings where the accuracy improvement of edge/cloud system over the standalone small edge DNN are set as {50.0%, 75.0%, 90.0%, 95.0%}, respectively. The relative

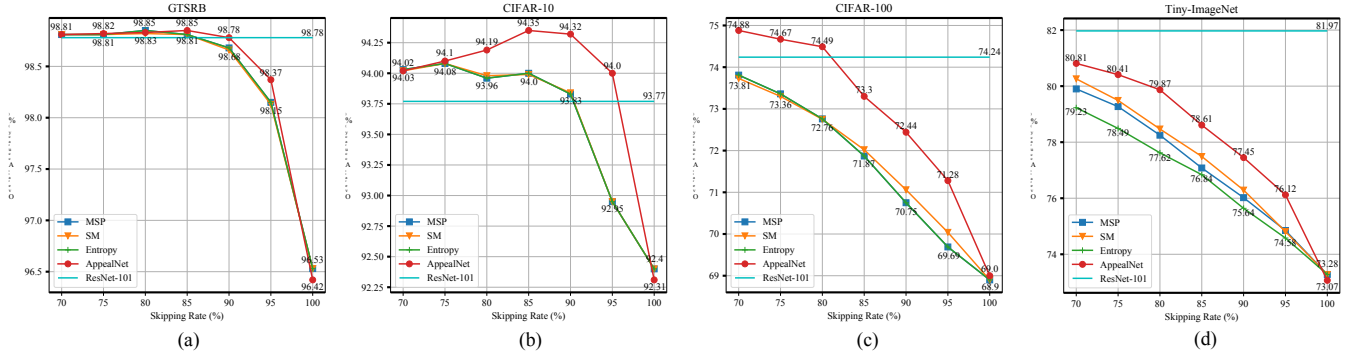


图 5. MobileNet 在 GTSRB、CIFAR10、CIFAR100 和 Tiny-ImageNet 上不同跳跃率下的总体准确率。

Dataset	Accuracy %	Cost MFLOPs	Cost at 50.0% AccI	Relative Saving at 50.0% AccI	Cost at 75.0% AccI	Relative Saving at 75.0% AccI	Cost at 90.0% AccI	Relative Saving at 90.0% AccI	Cost at 95.0% AccI	Relative Saving at 95.0% AccI
	ResNet-101/MobileNet/AppealNet		(SM/AppealNet)		(SM/AppealNet)		(SM/AppealNet)		(SM/AppealNet)	
GTSRB	98.78/96.53/96.42	2520.3/28.8/28.8	122.23/100.80	17.53%	192.24/123.48	35.77%	228.37/166.58	27.07%	227.20/191.99	15.50%
CIFAR-10	93.77/92.40/92.31	2520.3/94.6/94.6	217.10/138.02	36.43%	245.72/161.31	34.35%	291.32/164.70	43.46%	314.13/167.86	46.56%
CIFAR-100	74.24/68.90/69.00	2520.4/94.7/94.7	415.38/234.66	43.51%	634.41/385.54	39.22%	819.50/462.19	43.60%	884.51/513.89	41.90%
Tiny-ImageNet	81.97/73.28/73.07	7832.34/313.11/313.11	1482.35/1067.29	28.00%	2215.48/1724.47	22.16%	3089.21/2646.33	14.34%	4374.25/3835.71	12.31%

表一 不同精度要求下边缘/云架构的总体计算成本。

B. Visualization of Input Differentiation

在图 4 中，我们将 MSP 分数和 $q(z|x)$ 分数可视化，其中 EfficientNet 作为小型边缘网络，训练数据集为 CIFAR-10。correct 和 incorrect 输入分别指的是那些可以在边缘正确处理的输入和那些不能处理的输入。这两张图清楚地展示了 AppealNet 的优势。也就是说，AppealNet 能够在 $q(z|x)$ 分数方面很好地区分“简单”输入和“困难”输入，而 MSP 分数¹ 导致这两种输入之间有更多的重叠。这种优势是通过 AppealNet 架构中定制的预测器头以及具有理论基础的相应训练策略获得的。其他数据集和模型也观察到类似的现象。

C. Effectiveness of AppealNet

在本小节中，我们在边缘/云协作系统准确性（方程（13）中定义）和节能（例如方程（11）中定义的跳跃率或方程（15）中定义的计算成本）之间的权衡方面证明了 AppealNet 的有效性。

首先，我们在图 5² 中显示了不同跳跃率下的系统精度。从图中可以看出，在大多数情况下，在所有三个数据集上，AppealNet（红线）的准确性都高于基线方法。这样的结果符合我们的预期，因为 AppealNet 可以比基准解决方案更好地区分“简单”和“困难”输入。以图 5 (b) 为例，当跳跃率为 95% (i.e., 5% 的输入被卸载到复杂网络) 时，AppealNet 的系统准确率为 94.0%，而基准解决方案只能达到 92.95%。我们还可以看到，随着跳跃率的增加（在达到 100% 之前），AppealNet 相对于基准解决方案的相对优势（红线和其他线之间的差距）通常会更高。这是因为，当跳跃率较高时，这种区分能力的差异发挥着更重要的作用。

¹ISM 分数和 Entropy 分数的结果与 MSP 分数相似。

²这里，我们仅显示使用 MobileNet 作为小型边缘网络时的结果。我们在 EfficientNet 和 ShuffleNet 中看到了类似的结果。

人们可能会认为边缘/云系统的整体精度会低于独立的大型网络，因为小型网络容量较低。图 5 中的一个看似令人惊讶的观察结果是，边缘/云系统设计可以实现精度提升。除了 Tiny-ImageNet 之外，边缘/云系统的整体精度在某些跳跃率范围内可以高于单个 ResNet-101（虚线）的精度。这是因为，尽管小型网络对整个数据集的精度较低，但它与大网络共同优化以最小化总体预期损失（如式（9）所示），并且它很可能正确预测大网络无法正确分类的部分输入。与其他基于置信度的方法相比，对于更困难的 CIFAR-100 数据集，只有 AppealNet 可以产生比独立大型网络更高的准确性，如图 5 (c) 所示。对于 Tiny-ImageNet，当跳跃率在 [70, 100] 范围内时，大型网络和小网络之间的精度差距太大（超过 8%），无法实现精度提升。在这种情况下，我们仍然可以从 AppealNet 中受益，与其他基线解决方案相比，它在预测准确性和节能之间提供了更好的权衡。

正如我们观察到的，在大多数情况下，分数裕度基线的表现略好于其他两个基线，在后面的结果中，我们只将 AppealNet 与它进行比较。

表 I 通过计算在不同精度改进（在方程（14）中定义）下以 MFLOP 为单位测量的总体计算成本（在方程（15）中定义），展示了基线方法和 AppealNet 之间的比较。表的左侧部分分别展示了原始 ResNet-101、MobileNet 和 AppealNet 的准确性和计算成本。AppealNet 和 MobileNet 之间的主要区别在于 AppealNet 有一个额外的预测器头，并通过联合训练方案进行了优化（参见第 V-B 节），并且双头 AppealNet 的准确性接近 MobileNet 的准确性（参见表 I 中的第二列）。

我们现在转向表 I 的右侧部分，它显示了系统计算成本，其中阈值 δ 已调整为匹配目标系统精度。我们评估了四种设置，其中边缘/云系统相对于独立小边缘 DNN 的精度提升分别设置为 {50.0%、75.0%、90.0%、95.0%}。亲戚

Model	Original Accuracy (%)	AppealNet Accuracy (%)	AR at 50.0% AccI (SM/AppealNet)	Relative Saving at 50.0% AccI	AR at 75.0% AccI (SM/AppealNet)	Relative Saving at 75.0% AccI	AR at 90.0% AccI (SM/AppealNet)	Relative Saving at 90.0% AccI	AR at 95.0% AccI (SM/AppealNet)	Relative Saving at 95.0% AccI
EfficientNet	90.44	91.95	13.78/7.65	44.48%	25.30/18.63	26.36%	23.24/18.94	18.50%	36.53/44.36	17.65%
MobileNet	92.40	92.31	6.22/4.18	32.80%	12.49/16.74	25.39%	18.94/24.51	22.72%	25.59/34.59	26.02%
ShuffleNet	87.30	87.35	21.29/16.43	22.82%	34.95/28.27	19.11%	45.30/36.92	18.50%	45.59/54.37	16.15%

TABLE II

APPEALING RATE OF BLACK-BOX APPROXIMATION UNDER DIFFERENT ACCURACY REQUIREMENTS ON CIFAR-10

cost savings of AppealNet over score margin baseline at different accuracy degradation settings are listed in the table. We can see that AppealNet outperforms in all these cases by a large margin. That is, we can further reduce the computation cost of edge/cloud collaborative system on GTSRB, CIFAR-10, CIFAR-100 and Tiny-ImageNet up to 35.77%, 46.56%, 43.60% and 28.0%, respectively.

D. Black-Box Approximation

We further investigate the effectiveness of AppealNet under black-box settings, where the complex network is regarded as an oracle function without revealing its information. Since the oracle function always predicts correct results, the improvement of the overall accuracy comes from appealing those inputs that cannot be correctly classified by the small DNN to the cloud. As we cannot explicitly evaluate the computational cost of the whole system, the appealing rate is instead reported under accuracy improvement $AccI \in \{50.0\%, 75.0\%, 90.0\%, 95.0\%\}$. A lower appealing rate means less computations on the cloud and less communication cost, thereby saving more energy. As shown in Table II, three efficient DNN models and their AppealNet counterparts are constructed and evaluated. The results in the right part of Table II show the advantage of AppealNet over the baseline. For example, the relative saving of AppealNet on CIFAR-10 dataset under the black-box setting is 44.48% at 50% $AccI$ with EfficientNet as the small edge DNN architecture.

VII. CONCLUSION

In this paper, we propose AppealNet, an efficient and highly-accurate edge/cloud collaborative architecture for DNN-based classifiers. By employing a two-head architecture for the small edge network and jointly optimizing the approximator and the predictor in training, AppealNet can achieve trustworthy differentiation between easy and difficult inputs for the system, realizing up to more than 40% energy savings compared to state-of-the-art solutions.

VIII. ACKNOWLEDGEMENT

This work is supported in part by General Research Fund (GRF) of Hong Kong Research Grants Council (RGC) under Grant No. 14205018 and No. 14205420, and in part by National Natural Science Foundation of China (NSFC) under Grant No. 61834006.

REFERENCES

- [1] Y. He, X. Zhang, and J. Sun, "Channel pruning for accelerating very deep neural networks," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 1389–1397.
- [2] S. Han and et. al., "Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding," in *4th International Conference on Learning Representations (ICLR)*, 2016.
- [3] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," *arXiv preprint arXiv:1503.02531*, 2015.
- [4] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," *arXiv preprint arXiv:1704.04861*, 2017.
- [5] H. Tann, S. Hashemi, and et. al., "Runtime configurable deep neural networks for energy-accuracy trade-off," in *2016 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ ISSS)*. IEEE, 2016, pp. 1–10.
- [6] E. Park and et. al., "Big/little deep neural network for ultra low power inference," in *2015 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*. IEEE, 2015.
- [7] T. Bolukbasi and et. al., "Adaptive neural networks for efficient inference," in *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. JMLR. org, 2017, pp. 527–536.
- [8] D. Stamoulis, T.-W. Chin, A. K. Prakash, H. Fang, S. Sajja, M. Bognar, and D. Marculescu, "Designing adaptive neural networks for energy-constrained image classification," in *Proceedings of the International Conference on Computer-Aided Design*, 2018, pp. 1–8.
- [9] D. Hendrycks and K. Gimpel, "A baseline for detecting misclassified and out-of-distribution examples in neural networks," in *5th International Conference on Learning Representations, ICLR 2017*, 2017.
- [10] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. JMLR. org, 2017.
- [11] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in neural information processing systems*, 2017, pp. 6402–6413.
- [12] X. Wang, F. Yu, and et. al., "Skipnet: Learning dynamic routing in convolutional networks," in *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018, pp. 409–424.
- [13] Y. Wang, J. Shen, and et. al., "Dual dynamic inference: Enabling more efficient, adaptive and controllable deep inference," *IEEE Journal of Selected Topics in Signal Processing*, 2020.
- [14] P. Panda, A. Sengupta, and K. Roy, "Conditional deep learning for energy-efficient and enhanced pattern recognition," in *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2016.
- [15] Y. Long, I. Chakraborty, and K. Roy, "Conditionally deep hybrid neural networks across edge and cloud," *arXiv:2005.10851*, 2020.
- [16] S. Teerapittayanon, B. McDanel, and H.-T. Kung, "Branchynet: Fast inference via early exiting from deep neural networks," in *23rd International Conference on Pattern Recognition (ICPR)*. IEEE, 2016.
- [17] L. Liu and J. Deng, "Dynamic deep neural networks: Optimizing accuracy-efficiency trade-offs by selective execution," in *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018.
- [18] Y. Kaya and et. al., "Shallow-deep networks: Understanding and mitigating network overthinking," in *Proceedings of the 36th International Conference on Machine Learning, ICML*. PMLR, 2019.
- [19] L. Yang, Y. Han, and et. al., "Resolution adaptive networks for efficient inference," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020, pp. 2369–2378.
- [20] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [21] F. Nan and V. Saligrama, "Adaptive classification for prediction under a budget," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017, pp. 4730–4740.
- [22] H. W. Kuhn and A. W. Tucker, "Nonlinear programming," in *Traces and emergence of nonlinear programming*. Springer, 2014, pp. 247–258.
- [23] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning, ICML 2019*, 2019.
- [24] X. Zhang, X. Zhou, M. Lin, and J. Sun, "Shufflenet: An extremely efficient convolutional neural network for mobile devices," in *IEEE Conference on Computer Vision and Pattern Recognition, CVPR*, 2018.
- [25] J. Stallkamp, M. Schlipsing, J. Salmen, and C. Igel, "Man vs. computer: Benchmarking machine learning algorithms for traffic sign recognition," *Neural networks*, vol. 32, pp. 323–332, 2012.
- [26] A. Krizhevsky, G. Hinton et al., "Learning multiple layers of features from tiny images," Citeseer, Tech. Rep., 2009.
- [27] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei, "Imagenet: A large-scale hierarchical image database," in *2009 IEEE conference on computer vision and pattern recognition*. IEEE, 2009, pp. 248–255.

Model	Original Accuracy (%)	AppealNet Accuracy (%)	AR at 50.0% AccI (SM/AppealNet)	Relative Saving at 50.0% AccI	AR at 75.0% AccI (SM/AppealNet)	Relative Saving at 75.0% AccI	AR at 90.0% AccI (SM/AppealNet)	Relative Saving at 90.0% AccI	AR at 95.0% AccI (SM/AppealNet)	Relative Saving at 95.0% AccI
EfficientNet	90.44	91.95	13.78/7.65	44.48%	25.30/18.63	26.36%	23.24/18.94	18.50%	36.53/44.36	17.65%
MobileNet	92.40	92.31	6.22/4.18	32.80%	12.49/16.74	25.39%	18.94/24.51	22.72%	25.59/34.59	26.02%
ShuffleNet	87.30	87.35	21.29/16.43	22.82%	34.95/28.27	19.11%	45.30/36.92	18.50%	45.59/54.37	16.15%

表2 CIFAR-10不同精度要求下的黑盒逼近上诉率

表中列出了在不同准确度降级设置下 AppealNet 相对于分数余量基线所节省的成本。我们可以看到 AppealNet 在所有这些情况下都表现出色。也就是说，我们可以进一步将GTSRB、CIFAR-10、CIFAR-100和Tiny-ImageNet上的边缘/云协作系统的计算成本分别降低最多35.77%、46.56%、43.60%和28.0%。

D. Black-Box Approximation

我们进一步研究了 AppealNet 在黑盒设置下的有效性，其中复杂网络被视为预言机功能而不泄露其信息。由于预言机函数总是预测正确的结果，因此整体准确率的提高来自于将那些无法被小型 DNN 正确分类的输入吸引到云端。由于我们无法明确评估整个系统的计算成本，因此上诉率以准确度改进 $AccI \in \{50.0\%, 75.0\%, 90.0\%, 95.0\%\}$ 来报告。较低的吸引率意味着更少的云端计算和更少的通信成本，从而节省更多的能源。如表 II 所示，构建并评估了三个高效的 DNN 模型及其 AppealNet 模型。表 I 右侧的结果显示了 AppealNet 相对于基线的优势。例如，在黑盒设置下，以 EfficientNet 作为小边缘 DNN 架构，AppealNet 在 CIFAR-10 数据集上的相对节省量为 50% $AccI$ 时的 44.48%。

七. 结论

在本文中，我们提出了 AppealNet，这是一种用于基于 DNN 的分类器的高效且高精度的边缘/云协作架构。通过对小型边缘网络采用双头架构，并在训练中联合优化逼近器和预测器，AppealNet 可以在系统的简单输入和困难输入之间实现可信区分，与最先进的解决方案相比，实现高达 40% 以上的节能。

八. 致谢

这项工作部分得到香港研究资助局（RGC）一般研究基金（GRF）的资助，资助号为14205018和14205420，部分得到国家自然科学基金委员会（NSFC）的资助，资助号为61834006。

参考

[1] Y.He, X.Zhang 和 J.Sun, “加速超深度神经网络的通道剪枝”, 载于 *Proceedings of the IEEE International Conference on Computer Vision*, 2017 年, 第 1389-1397 页。[2] S. Han 等。等人, “深度压缩: 通过剪枝、训练量化和霍夫曼编码来压缩深度神经网络”, *4th International Conference on Learning Representations (ICLR)*, 2016 年。[3] G. Hinton, O. Vinyals 和 J. Dean, “提取神经网络中的知识”, *arXiv preprint arXiv:1503.02531*, 2015 年。[4] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto 和 H. Adam, “Mobilenets: 用于移动视觉应用的高效卷积神经网络”, *arXiv preprint arXiv:1704.04861*, 2017 年。[5] H. Tann, S. Hashemi 和 el. 等人, “用于能量精度权衡的运行时可配置深度神经网络”, *2016 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ ISSS)*. IEEE, 2016 年, 第 1-10 页。

[6] E. Park 等。等人, “用于超低功耗推理的大/小深度神经网络”, *2015 International Conference on Hardware/Software Codesign and System Synthesis (CODES+ISSS)*. IEEE, 2015。[7] T. Bolukbasi 等。等人, “用于高效推理的自适应神经网络”, *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. JMLR. 组织, 2017 年, 第 527–536 页。[8] D. Stamoulis, T.-W. Chin, A.K. Prakash, H. Fang, S. Sajja, M. Bognar 和 D. Marculescu, “设计用于能量约束图像分类的自适应神经网络”, *Proceedings of the International Conference on Computer-Aided Design*, 2018 年, 第 1-8 页。[9] D. Hendrycks 和 K. Gimpel, “检测神经网络中错误分类和分布外示例的基线”, 见 *5th International Conference on Learning Representations, ICLR 2017*, 2017。[10] C.Guo, G. Pleiss, Y. Sun 和 K. Q. Weinberger, “关于现代神经网络的校准”, 见 *Proceedings of the 34th International Conference on Machine Learning-Volume 70*. JMLR. org, 2017。[11] B. Lakshminarayanan, A. Pritzel 和 C. Blundell, “使用深度集成的简单且可扩展的预测不确定性估计”, *Advances in neural information processing systems*, 2017 年, 第 6402-6413 页。[12] 王X., 于F., 等。等人, “Skipnet: 学习卷积网络中的动态路由”, *Proceedings of the European Conference on Computer Vision (ECCV)*, 2018 年, 第 409-424 页。[13] 王勇, 沉杰, 等。al., “双重动态推理: 实现更高效、自适应和可控的深度推理”, *IEEE Journal of Selected Topics in Signal Processing*, 2020 年。[14] P. Panda, A. Sengupta 和 K. Roy, “用于节能和增强模式识别的条件深度学习”, *Design, Automation & Test in Europe Conference & Exhibition (DATE)*. IEEE, 2016。[15] Y. Long, I. Chakraborty 和 K. Roy, “跨边缘和云的有条件深度混合神经网络” *arXiv:2005.10851*, 2020。[16] S. Teerapittayanon, B. McDanel 和 H.-T. Kung, “Branchynet: 通过早期退出深度神经网络进行快速推理”, *23rd International Conference on Pattern Recognition (ICPR)*. IEEE, 2016。[17] L. Liu 和 J. Deng, “动态深度神经网络: 通过选择性执行优化准确性-效率权衡”, *Thirty-Second AAAI Conference on Artificial Intelligence*, 2018 年。[18] Y. Kaya 等人。等人, “浅层网络: 理解和减轻网络过度思考”, *Proceedings of the 36th International Conference on Machine Learning, ICML*. PMLR, 2019。[19] L. Yang, Y. Han, 等。等人, “用于高效推理的分辨率自适应网络”, *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2020 年, 第 2369-2378 页。[20] K. He, X. Zhang, S. Ren, J. Sun, “图像识别的深度残差学习”, 载于 *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016 年, 第 770-778 页。[21] F. Nan 和 V. Saligrama, “预算下预测的自适应分类”, *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017 年, 第 4730-4740 页。[22] H. W. Kuhn 和 A. W. Tucker, “非线性规划”, *Traces and emergence of nonlinear programming*. 施普林格, 2014 年, 第 247-258 页。[23] M. Tan 和 Q. V. Le, “Efficientnet: 重新思考卷积神经网络的模型扩展”, 载于 *Proceedings of the 36th International Conference on Machine Learning, ICML 2019*, 2019 年。[24] X. Zhang, X. Zhou, M. Lin 和 J. Sun, “Shufflenet: 一种用于移动设备的极其高效的卷积神经网络”, 载于 *IEEE Conference on Computer Vision and Pattern Recognition, CVPR*, 2018 年。[25] J. Stallkamp, M. Schlipsing, J. Salmen 和 C. Igel, “人与计算机: 交通标志识别机器学习算法的基准”, *Neural networks*, 卷. 32, 第 323–332 页, 2012 年。[26] A. Krizhevsky, G. Hinton *et al.*, “从微小图像中学习多层特征”, Citeseer, Tech. 代表, 2009。[27] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li 和 L. Fei-Fei, “Imagenet: 大规模分层图像数据库”, *2009 IEEE conference on computer vision and pattern recognition*. IEEE, 2009 年, 第 248–255 页。